LAB 3: HÀM - CON TRỎ - CẤP PHÁT ĐỘNG

I. Mục tiêu

Sau khi hoàn thành tài liệu này, sinh viên có thể:

- Hiểu và sử dụng hàm trong C, nắm được phạm vi biến, call stack và cơ chế đệ quy.
- Giải thích và vận dụng con trỏ trong truyền tham số, mảng và xâu ký tự.
- Sử dụng các hàm cấp phát bộ nhớ động (malloc, calloc, realloc, free).
- Nhận biết và tránh các lỗi bộ nhớ phổ biến như memory leak, dangling pointer, double free.
- Cài đặt và thao tác với danh sách liên kết đơn trong C.

II. Lý thuyết cơ bản

1. Hàm

1.1 Giới thiệu về hàm

Trong lập trình, khi chương trình trở nên phức tạp hơn, việc tổ chức và quản lý mã nguồn đóng vai trò rất quan trọng. Nếu toàn bộ chương trình được viết trong một khối lệnh duy nhất (thường là trong hàm main()), mã nguồn sẽ trở nên dài, khó đọc và khó bảo trì. Đây chính là lý do cần đến khái niệm hàm (function).

Để hiểu rõ vấn đề, ta xét một chương trình đơn giản tính diện tích của nhiều hình tròn:

```
#include <stdio.h>

int main() {
    double pi = 3.14159;

    double rA = 5.0;
    double areaA = pi * rA * rA;
    printf("A = %.2f\n", areaA);

    double rB = 3.0;
    double areaB = pi * rB * rB;
    printf("B = %.2f\n", areaB);
```

```
    double rC = 7.1;
    double areaC = pi * rC + rC; // lỗi công thức
    printf("C = %.2f\n", areaC);

    return 0;
}
```

Trong chương trình trên, công thức tính diện tích hình tròn được lặp lại nhiều lần. Việc lặp lại này dẫn đến một số vấn đề. Thứ nhất, nếu công thức cần thay đổi, người lập trình phải sửa ở nhiều vị trí khác nhau, dễ gây thiếu sót. Thứ hai, việc sao chép và chỉnh sửa code làm tăng nguy cơ xuất hiện lỗi, như trong ví dụ trên, công thức ở hình tròn thứ ba bị viết sai nhưng khó phát hiện. Thứ ba, chương trình trở nên khó đọc vì người đọc phải theo dõi toàn bộ các dòng lệnh để hiểu mục đích chung.

Để giải quyết những vấn đề này, ta sử dụng hàm nhằm đóng gói logic xử lý thành các đơn vị nhỏ, có thể tái sử dụng. Chương trình trên có thể được viết lại như sau:

```
#include <stdio.h>

double area(double r) {
    return 3.14159 * r * r;
}

int main() {
    printf("A = %.2f\n", area(5.0));
    printf("B = %.2f\n", area(3.0));
    printf("C = %.2f\n", area(7.1));
    return 0;
}
```

Trong phiên bản này, công thức tính diện tích chỉ được viết một lần trong hàm area(). Việc sử dụng hàm mang lại nhiều lợi ích. Chương trình trở nên dễ đọc hơn vì main() chỉ chứa các lời gọi hàm mang ý nghĩa rõ ràng. Việc kiểm thử cũng thuận tiện hơn, vì có thể

kiểm tra riêng từng hàm. Ngoài ra, khi cần thay đổi công thức, chỉ cần chỉnh sửa tại một vị trí duy nhất.

Như vậy, hàm là một công cụ quan trọng giúp tổ chức chương trình theo hướng rõ ràng, dễ hiểu và dễ bảo trì.

1.2 Khai báo hàm (Declaration)

Trong ngôn ngữ C, trước khi một hàm được sử dụng, trình biên dịch cần biết thông tin cơ bản về hàm đó, bao gồm tên hàm, kiểu dữ liệu trả về và danh sách tham số. Việc cung cấp thông tin này được gọi là khai báo hàm (function declaration hoặc function prototype).

Cú pháp tổng quát của khai báo hàm như sau:

kiểu_trả_về tên_hàm(danh_sách_tham_số);

Ví dụ:

double area(double r);

Dòng lệnh trên cho trình biên dịch biết rằng tồn tại một hàm tên là area, nhận vào một tham số kiểu double và trả về một giá trị kiểu double.

Việc khai báo hàm là cần thiết do đặc điểm biên dịch của ngôn ngữ C. Trình biên dịch xử lý chương trình theo thứ tự từ trên xuống dưới. Nếu một hàm được gọi trước khi được định nghĩa, trình biên dịch sẽ không biết cách xử lý lời gọi đó và sinh lỗi.

Xét ví dụ sau:

```

```

Trong trường hợp này, hàm area() được gọi trong main() trước khi nó được định nghĩa, dẫn đến lỗi biên dịch. Để khắc phục, ta cần khai báo hàm trước khi sử dụng:

```
#include <stdio.h>

int main() {
    printf("%.2f\n", area(5.0)); // lỗi
    return 0;
}
```

```
double area(double r) {
   return 3.14159 * r * r;
}
```

Khai báo hàm giúp trình biên dịch kiểm tra tính đúng đắn của lời gọi hàm, chẳng hạn như số lượng tham số và kiểu dữ liệu. Ngoài ra, việc sử dụng prototype còn giúp tăng tính rõ ràng và chuyên nghiệp của chương trình, đặc biệt trong các dự án lớn có nhiều file nguồn.

1.3 Định nghĩa hàm (Definition)

Định nghĩa hàm là phần cài đặt chi tiết chức năng của hàm, nơi chứa các câu lệnh thực hiện nhiệm vụ cụ thể. Nếu khai báo hàm chỉ cung cấp thông tin tổng quát, thì định nghĩa hàm chính là nội dung thực sự của hàm đó.

Một hàm trong C có cấu trúc chung như sau:

```
kiểu_trả_về tên_hàm(danh_sách_tham_số) {
   // các câu lệnh xử lý
   return giá_trị;
}
```

Mỗi thành phần trong cấu trúc này đều có vai trò riêng.

Kiểu trả về (return type) xác định loại dữ liệu mà hàm sẽ trả về sau khi thực hiện xong. Ví dụ, trong hàm area(), kiểu trả về là double, cho biết kết quả là một số thực. Trong trường hợp hàm không trả về giá trị, ta sử dụng kiểu void.

Tên hàm (function name) là định danh được sử dụng để gọi hàm. Việc đặt tên hàm nên phản ánh đúng chức năng của hàm, giúp chương trình dễ đọc và dễ hiểu hơn.

Danh sách tham số (parameters) là các biến nhận giá trị từ bên ngoài khi hàm được gọi. Trong C, các tham số này được truyền theo cơ chế truyền giá trị (pass-by-value), nghĩa là hàm sẽ làm việc với bản sao của dữ liệu. Do đó, việc thay đổi giá trị của tham số bên trong hàm sẽ không ảnh hưởng đến biến ban đầu.

Ví dụ:

```
#include <stdio.h>

void change(int x) {
```

```
    x = 100;
}

int main() {
    int a = 5;
    change(a);
    printf("%d\n", a); // vẫn là 5
    return 0;
}
```

Trong ví dụ này, biến a không bị thay đổi sau khi gọi hàm change() vì x chỉ là bản sao của a.

Phần thân hàm (body) nằm trong cặp dấu ngoặc { }, chứa các câu lệnh thực hiện xử lý. Đây là nơi ta có thể khai báo biến cục bộ, thực hiện phép toán, sử dụng cấu trúc điều khiển, v.v.

Câu lệnh return được sử dụng để trả kết quả về cho nơi gọi hàm và kết thúc việc thực thi hàm. Khi gặp return, chương trình sẽ thoát khỏi hàm ngay lập tức.

Để minh họa đầy đủ, ta xét ví dụ sau:

```
#include <stdio.h>

int square(int x) {
    int result = x * x;
    return result;
}

int main() {
    int a = 4;
    int b = square(a);
    printf("Square = %d\n", b);
    return 0;
}
```

Trong chương trình này, hàm square() nhận một số nguyên làm đầu vào, tính bình phương của số đó và trả về kết quả. Hàm main() gọi hàm square() và in kết quả ra màn hình.

1.4 Biến toàn cục và biến cục bộ

Trong một chương trình C, biến có thể được khai báo ở nhiều vị trí khác nhau, và vị trí khai báo sẽ quyết định phạm vi sử dụng của biến đó. Khái niệm phạm vi (scope) giúp xác định nơi mà một biến có thể được truy cập và sử dụng. Hai loại biến quan trọng nhất là biến toàn cục (global variable) và biến cục bộ (local variable).

Việc hiểu rõ phạm vi của biến không chỉ giúp tránh lỗi trong chương trình mà còn giúp tổ chức mã nguồn một cách rõ ràng và an toàn hơn.

1.4.1 Biến toàn cục

Biến toàn cục là biến được khai báo bên ngoài tất cả các hàm trong chương trình. Những biến này có thể được truy cập từ bất kỳ hàm nào trong cùng một file nguồn.

Ví dụ:

```
#include <stdio.h>

int g = 0; // biến toàn cục

void update() {
   g = g + 10;
}

int main() {
   printf("Before: %d\n", g);
   update();
   printf("After: %d\n", g);
   return 0;
}
```

Trong chương trình trên, biến g được khai báo bên ngoài các hàm, nên cả main() và update() đều có thể truy cập và thay đổi giá trị của biến này.

Biến toàn cục tồn tại trong suốt thời gian chạy của chương trình. Điều này có nghĩa là giá trị của nó không bị mất đi sau khi một hàm kết thúc. Đây là một đặc điểm khác biệt so với biến cục bộ.

Tuy nhiên, việc sử dụng biến toàn cục cần được cân nhắc cẩn thận. Khi nhiều hàm cùng truy cập và thay đổi một biến toàn cục, chương trình có thể trở nên khó kiểm soát và khó phát hiện lỗi. Trong các chương trình lớn, việc lạm dụng biến toàn cục thường dẫn đến lỗi khó debug và làm giảm tính mô-đun của chương trình.

1.4.2 Biến cục bộ

Biến cục bộ là biến được khai báo bên trong một hàm hoặc một khối lệnh. Phạm vi sử dụng của biến này chỉ giới hạn trong khu vực mà nó được khai báo.

Xét ví dụ sau:

```
#include <stdio.h>

void foo() {
   int x = 10;  // biến cục bộ
   printf("x in foo = %d\n", x);
}

int main() {
   foo();
   // printf("%d\n", x); // lỗi: x không tồn tại ở đây
   return 0;
}
```

Trong ví dụ này, biến x chỉ tồn tại trong hàm foo(). Khi hàm kết thúc, biến x sẽ bị hủy và không thể truy cập từ bên ngoài. Một điểm quan trọng là mỗi lần hàm được gọi, các biến cục bộ sẽ được tạo mới. Điều này đảm bảo rằng các lần gọi hàm là độc lập với nhau.

Bên cạnh biến cục bộ thông thường, ngôn ngữ C còn cung cấp biến static, cho phép kết hợp giữa phạm vi cục bộ và thời gian sống dài hơn.

```
#include <stdio.h>

void counter() {
```

```
    static int count = 0; // biến static
    count++;
    printf("Count = %d\n", count);
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Trong ví dụ này, biến count chỉ có thể được truy cập bên trong hàm counter(), nhưng giá trị của nó không bị mất sau mỗi lần gọi hàm. Điều này cho thấy biến static có phạm vi cục bộ nhưng thời gian sống giống biến toàn cục.

Ngoài phạm vi của hàm, trong C còn tồn tại phạm vi của khối lệnh (block scope). Một biến được khai báo trong cặp dấu { ... } chỉ tồn tại trong khối đó.

```
#include <stdio.h>

int main() {
    int a = 5;

    {
        int b = 10;
        printf("a = %d, b = %d\n", a, b);
    }

    // printf("%d\n", b); // lỗi: b không tồn tại ở đây
    return 0;
}
```

Trong ví dụ trên, biến b chỉ tồn tại trong khối { ... }. Khi chương trình thoát khỏi khối này, biến b không còn tồn tại.

Một đặc điểm quan trọng của biến cục bộ là các biến trong các hàm khác nhau không ảnh hưởng lẫn nhau, ngay cả khi chúng có cùng tên. Điều này giúp tránh xung đột dữ liệu và làm cho chương trình dễ quản lý hơn.

```
#include <stdio.h>

void func1() {
   int x = 10;
   printf("func1 x = %d\n", x);
}

void func2() {
   int x = 20;
   printf("func2 x = %d\n", x);
}

int main() {
   func1();
   func2();
   return 0;
}
```

Mặc dù cả hai hàm đều khai báo biến x, nhưng chúng là hai biến hoàn toàn độc lập. Việc thay đổi x trong func1() không ảnh hưởng đến x trong func2().

Bên cạnh đó, chỉ nên khai báo biến ở phạm vi nhỏ nhất cần dùng.

Mã nguồn này tốt hơn

```
for (int i = 0; i < 10; i++) {
   printf("%d\n", i);
}
```

mã nguồn dưới đây:

```
int i;
for (i = 0; i < 10; i++) {
   printf("%d\n", i);
}
```

Nguyên nhân do biến i không cần tồn tại ngoài vòng lặp.

Việc sử dụng phạm vi biến một cách hợp lý mang lại nhiều lợi ích quan trọng. Trước hết, nó giúp bảo vệ dữ liệu khỏi việc bị thay đổi ngoài ý muốn. Một biến cục bộ chỉ có thể được truy cập trong phạm vi xác định, do đó các phần khác của chương trình không thể can thiệp trực tiếp vào nó. Điều này giúp giảm thiểu lỗi và tăng độ tin cậy của chương trình.

Bên cạnh đó, việc sử dụng biến cục bộ còn giúp chương trình trở nên rõ ràng hơn, vì mỗi hàm chỉ làm việc với dữ liệu của riêng nó. Đây là một nguyên tắc quan trọng trong lập trình có cấu trúc, giúp xây dựng các chương trình dễ hiểu, dễ kiểm thử và dễ bảo trì.

1.5 Call stack và stack frame

Khi một chương trình C thực thi, đặc biệt là khi có nhiều hàm được gọi lồng nhau, hệ thống cần một cơ chế để quản lý thứ tự thực thi và lưu trữ thông tin tạm thời của từng hàm. Cơ chế này được thực hiện thông qua ngăn xếp lời gọi (call stack) và các khung ngăn xếp (stack frame).

Hiểu rõ cách hoạt động của call stack giúp giải thích nhiều hiện tượng quan trọng trong lập trình như: tại sao biến cục bộ bị mất sau khi hàm kết thúc, cách hàm quay trở lại vị trí gọi ban đầu, hay nguyên nhân gây ra lỗi stack overflow trong đệ quy.

1.5.1 Stack frame

Mỗi khi một hàm được gọi, hệ thống sẽ cấp phát một vùng nhớ riêng trên stack để lưu trữ thông tin phục vụ cho việc thực thi hàm đó. Vùng nhớ này được gọi là stack frame (khung ngăn xếp). Có thể hiểu một cách đơn giản, stack frame là không gian làm việc riêng của một lần gọi hàm.

Một stack frame thường chứa các thành phần quan trọng sau:

- Tham số của hàm: là bản sao của các giá trị được truyền vào
- Biến cục bộ: các biến được khai báo trong thân hàm

- Địa chỉ trả về (return address): vị trí trong chương trình mà hàm sẽ quay lại sau khi thực thi xong
- Thanh ghi được lưu (saved registers): trạng thái của CPU cần khôi phục sau khi hàm kết thúc

Xét ví dụ sau:

```
#include <stdio.h>

void foo() {
   int x = 10;
   printf("x = %d\n", x);
}

int main() {
   foo();
   return 0;
}
```

Khi hàm foo() được gọi, một stack frame mới được tạo ra để lưu biến x và các thông tin liên quan. Khi foo() kết thúc, stack frame này sẽ bị hủy và biến x không còn tồn tại nữa. Đây chính là lý do tại sao biến cục bộ chỉ tồn tại trong phạm vi của hàm.

Một hệ quả rất quan trọng là: không được trả về địa chỉ của biến cục bộ. Ví dụ, nếu một hàm tạo ra biến int x = 10; rồi trả về &x, thì địa chỉ đó trỏ vào vùng nhớ thuộc frame của hàm. Nhưng ngay khi hàm kết thúc, frame ấy bị xóa khỏi call stack, nên địa chỉ vừa trả về trở thành địa chỉ không còn hợp lệ. Đây là lỗi kinh điển dẫn đến dangling pointer.

1.5.2 Call stack

Call stack là một cấu trúc dữ liệu dạng ngăn xếp (stack) được sử dụng để quản lý các stack frame của các hàm đang hoạt động. Nguyên tắc hoạt động của call stack là LIFO (Last In, First Out), nghĩa là hàm được gọi sau cùng sẽ được kết thúc trước.

Xét ví dụ sau:

```
#include <stdio.h>
```

```
void bar() {
   printf("In bar()\n");
}

void foo() {
   bar();
}

int main() {
   foo();
   return 0;
}
```

Quá trình thực thi của chương trình có thể được mô tả như sau:

- Khi chương trình bắt đầu, hàm main() được đưa vào stack
- main() gọi foo() → một stack frame mới của foo() được đẩy (push) vào stack
- foo() gọi bar() → stack frame của bar() tiếp tục được đẩy vào stack
- Sau khi bar() thực thi xong, stack frame của bar() bị loại bỏ (pop)
- Chương trình quay lại foo(), sau đó foo() kết thúc và frame của nó cũng bị pop
- Cuối cùng, main() kết thúc và stack trở về trạng thái ban đầu

Một yếu tố quan trọng trong việc quản lý call stack là hai con trỏ:

Stack pointer (SP) là con trỏ trỏ đến đỉnh hiện tại của stack. Khi một hàm mới được gọi, SP sẽ di chuyển để cấp phát không gian cho stack frame mới. Khi hàm kết thúc, SP sẽ quay trở lại vị trí trước đó.

Frame pointer (FP) là con trỏ dùng để đánh dấu vị trí bắt đầu của stack frame hiện tại. FP giúp truy cập dễ dàng đến các biến cục bộ và tham số của hàm, ngay cả khi stack pointer thay đổi trong quá trình thực thi.

Nhờ có SP và FP, hệ thống có thể quản lý nhiều hàm lồng nhau một cách chính xác và hiệu quả.

1.6 Đệ quy

Trong lập trình, bên cạnh việc sử dụng vòng lặp để giải quyết các bài toán lặp lại, ta còn có một kỹ thuật khác mang tính khái niệm cao hơn, đó là đệ quy (recursion). Đệ quy là một công cụ mạnh mẽ, đặc biệt hữu ích khi làm việc với các bài toán có cấu trúc lặp lại theo dạng tự giống chính nó, chẳng hạn như tính giai thừa, duyệt cây, hay tìm kiếm nhị phân.

Đệ quy là kỹ thuật trong đó một hàm tự gọi lại chính nó để giải quyết một bài toán. Thay vì giải trực tiếp bài toán lớn, ta chia nó thành các bài toán nhỏ hơn cùng dạng, và tiếp tục giải các bài toán nhỏ đó cho đến khi đạt được trường hợp đơn giản nhất.

Một ví dụ kinh điển của đệ quy là tính giai thừa:

```
#include <stdio.h>

int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    printf("%d\n", factorial(4)); // 4! = 24
    return 0;
}
```

Trong ví dụ này, lời gọi factorial(4) sẽ được phân rã thành:

- factorial(4) = 4 * factorial(3)
- factorial(3) = 3 * factorial(2)
- factorial(2) = 2 * factorial(1)
- factorial(1) = 1 (trường hợp dừng)

Sau đó, kết quả được tính ngược trở lại.

Một hàm đệ quy đúng phải luôn có hai thành phần bắt buộc.

Thành phần thứ nhất là trường hợp dừng (base case). Đây là điều kiện giúp hàm dừng việc gọi lại chính nó. Nếu không có base case, hàm sẽ tiếp tục gọi vô hạn.

Thành phần thứ hai là bước đệ quy (recursive step). Đây là phần mà hàm gọi lại chính nó với một đầu vào nhỏ hơn hoặc gần với base case hơn. Việc này đảm bảo rằng sau một số hữu hạn bước, chương trình sẽ đạt đến base case.

Trong ví dụ trên, điều kiện n <= 1 chính là base case, còn lời gọi factorial(n - 1) là bước đệ quy giúp giảm dần giá trị của n. Nếu một trong hai thành phần này bị thiếu hoặc sai, chương trình có thể hoạt động không đúng hoặc không bao giờ dừng.

Khi sử dụng đệ quy, mỗi lần hàm gọi lại chính nó sẽ tạo ra một stack frame mới trên call stack. Điều này có nghĩa là đệ quy tiêu tốn bộ nhớ stack tương ứng với số lần gọi hàm. Nếu số lượng lời gọi đệ quy quá lớn, vùng nhớ stack có thể bị đầy, dẫn đến lỗi stack overflow. Stack overflow là tình trạng khi chương trình sử dụng vượt quá dung lượng stack được cấp phát, khiến chương trình bị dừng đột ngột (thường kèm theo lỗi segmentation fault).

Xét ví dụ sau:

```
int factorial(int n) {
   return n * factorial(n - 1); // thiếu base case
}
```

Trong trường hợp này, hàm không có điều kiện dừng. Khi gọi factorial(4), chương trình sẽ tiếp tục gọi:

- factorial(3)
- factorial(2)
- factorial(1)
- factorial(0)
- factorial(-1)
- ...

Quá trình này không bao giờ kết thúc, dẫn đến việc call stack liên tục tăng lên cho đến khi bị tràn.

Ngoài trường hợp thiếu base case, stack overflow còn có thể xảy ra khi:

- Độ sâu đệ quy quá lớn (ví dụ: gọi đệ quy với n rất lớn)
- Bước đệ quy không làm giảm bài toán (không tiến gần base case)
- Sử dụng đệ quy trong các bài toán có thể lặp nhiều lần không cần thiết

Do đó, khi sử dụng đệ quy, cần đảm bảo rằng bài toán được thu nhỏ đúng cách và số lần gọi hàm là hợp lý.

2. Con trỏ (Pointers)

Trong ngôn ngữ C, con trỏ (pointer) là một trong những khái niệm quan trọng và có ảnh hưởng sâu sắc đến cách chương trình làm việc với bộ nhớ (memory). Khác với nhiều ngôn ngữ bậc cao, C cho phép lập trình viên truy cập trực tiếp đến địa chỉ bộ nhớ, từ đó mang lại hiệu năng cao và khả năng kiểm soát chi tiết, nhưng đồng thời cũng tiềm ẩn nhiều rủi ro nếu sử dụng không đúng cách.

Phần này sẽ trình bày một cách có hệ thống từ cơ chế truyền tham số trong C đến việc sử dụng con trỏ với mảng, số học con trỏ và xâu ký tự.

2.1 Pass by value

Trong C, tất cả các tham số được truyền vào hàm đều theo cơ chế truyền giá trị (pass by value). Điều này có nghĩa là khi gọi một hàm, giá trị của biến được sao chép (copy) vào tham số của hàm. Hàm sẽ làm việc với bản sao này thay vì biến gốc.

Để minh họa rõ hơn, ta xét bài toán hoán đổi hai số:

```
#include <stdio.h>

void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(x, y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Kết quả của chương trình:

x = 5, y = 10

Mặc dù trong hàm swap(), các giá trị của a và b đã được hoán đổi, nhưng sự thay đổi này không ảnh hưởng đến x và y trong hàm main().

Nguyên nhân nằm ở cơ chế pass by value. Khi gọi swap(x, y), hệ thống tạo ra hai biến mới a và b trong stack frame của hàm swap(), với giá trị lần lượt là 5 và 10. Mọi thao tác trong hàm chỉ ảnh hưởng đến các biến này. Khi hàm kết thúc, stack frame bị hủy và các thay đổi không được giữ lại.

2.2 Giới thiệu về con trỏ

Con trỏ (pointer) là một biến dùng để lưu trữ địa chỉ bộ nhớ (memory address) của một biến khác. Thay vì lưu giá trị trực tiếp, con trỏ cho phép ta truy cập gián tiếp thông qua địa chỉ.

Khai báo con trỏ (Pointer declaration)

Cú pháp khai báo con trỏ:

kiểu_dữ_liệu *tên_con_trỏ;

Ví dụ:

```
int *p;     // con trỏ tới int
double *q;  // con trỏ tới double
char *c;    // con trỏ tới char
```

Dấu * cho biết biến đó là con trỏ, và kiểu dữ liệu đi kèm xác định kiểu của dữ liệu mà con trỏ trỏ tới.

Khởi tạo con trỏ (Pointer initialization)

Sau khi khai báo, con trỏ cần được khởi tạo trước khi sử dụng. Có nhiều cách khởi tạo:

1. Gán địa chỉ của biến:

```
int x = 10;
int *p = &x;
```

2. Gán giá trị NULL:

```
int *p = NULL;
```

Đây là cách khởi tạo an toàn khi con trỏ chưa trỏ đến vùng nhớ hợp lệ.

3. Khởi tạo động (sẽ được giới thiệu ở phần sau):

```
int *p = malloc(sizeof(int));
```

Các toán tử quan trọng

Hai toán tử cơ bản khi làm việc với con trỏ là:

- Toán tử & (address-of): lấy địa chỉ của biến
- Toán tử * (dereference): truy cập giá trị tại địa chỉ

Ví dụ:

```
int x = 10;
int *p = &x;

printf("%d\n", *p); // in ra 10
```

Quan hệ quan trọng:

```
*p == x
*(&x) == x
```

Phân tích lại ví dụ swap (pass by reference)

Bằng cách truyền địa chỉ vào hàm, ta có thể thay đổi trực tiếp giá trị của biến:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Trong trường hợp này:

- &x, &y là địa chỉ
- a, b là con trỏ
- *a, *b thao tác trực tiếp trên dữ liệu gốc

Trong trường hợp này:

- &x và &y truyền địa chỉ của biến
- a và b là các con trỏ trỏ đến x và y
- *a và *b truy cập trực tiếp giá trị của x và y

Kết quả:

x = 10, y = 5

Ví dụ này cho thấy con trỏ cho phép thao tác trực tiếp trên dữ liệu gốc, điều mà pass by value không thể thực hiện được.

Một số lỗi thường gặp với con trỏ

- Con trỏ chưa khởi tạo (wild pointer): trỏ đến vùng nhớ không xác định
- Dereference NULL pointer: gây crash chương trình
- Dangling pointer: con trỏ trỏ đến vùng nhớ đã bị giải phóng

Ví dụ lỗi:

```
int *p;
*p = 10; // lỗi nghiêm trọng
```

2.3 Con trỏ và mảng

Trong C, mảng (array) và con trỏ có mối quan hệ rất chặt chẽ. Khi sử dụng tên mảng trong biểu thức, nó thường tự động chuyển thành con trỏ đến phần tử đầu tiên (hiện tượng array decay).

Ví dụ:

```
#include <stdio.h>

int main() {
    int arr[4] = {10, 20, 30, 40};
```

```
    printf("%d\n", *arr);       // 10
    printf("%d\n", arr[1]);     // 20

    int *p = arr;
    printf("%d\n", p[1]);       // 20
    printf("%d\n", *(p + 1));   // 20

    return 0;
}
```

Trong ví dụ này:

- arr tương đương với &arr[0]
- p[1] tương đương với *(p + 1)
- arr[i] tương đương với *(arr + i)

Điều này cho thấy việc truy cập mảng thực chất là sử dụng con trỏ.

Tuy nhiên, cần phân biệt rõ:

- Tên mảng là một địa chỉ cố định (không thể thay đổi)
- Con trỏ là biến (có thể thay đổi giá trị)

Ví dụ sau sẽ gây lỗi:

// arr++; // sai: không thể thay đổi địa chỉ của mảng

2.4 Số học trên con trỏ (Pointer arithmetic)

Con trỏ trong C hỗ trợ các phép toán số học như cộng, trừ và so sánh. Tuy nhiên, các phép toán này không hoạt động theo đơn vị byte mà theo kích thước của kiểu dữ liệu.

Ví dụ:

```
#include <stdio.h>

int main() {
    int arr[4] = {10, 20, 30, 40};
    int *p = arr;
```

```
    p = p + 2;
    printf("%d\n", *p); // 30

    return 0;
}
```

Ở đây, p + 2 sẽ dịch chuyển con trỏ đến phần tử thứ 3 của mảng, tương đương với việc tăng 2 * sizeof(int) byte trong bộ nhớ.

Ngoài ra, ta có thể thực hiện phép trừ giữa hai con trỏ:

int *q = &arr[4];
printf("%ld\n", q - p);

Kết quả cho biết số phần tử giữa hai con trỏ, không phải số byte.

Các phép so sánh như <, >, == cũng có thể áp dụng với con trỏ, nhưng chỉ hợp lệ khi các con trỏ trỏ vào cùng một mảng.

Một điểm rất quan trọng là nếu con trỏ vượt ra ngoài phạm vi hợp lệ của mảng, chương trình sẽ rơi vào trạng thái hành vi không xác định (undefined behavior), có thể dẫn đến lỗi hoặc kết quả sai.

2.5 Con trỏ với xâu ký tự (Strings)

Trong C, xâu ký tự (string) thực chất là một mảng các ký tự (char) kết thúc bằng ký tự đặc biệt '\0'. Con trỏ được sử dụng rất phổ biến để thao tác với chuỗi.

```
#include <stdio.h>

int main() {
    char name[] = "Alex";
    char *p = name;

    while (*p != '\0') {
        printf("%c", *p);
        p++;
    }
```

```
    return 0;
}
```

Ví dụ:

Chương trình trên sử dụng con trỏ để duyệt từng ký tự trong chuỗi.

Một điểm quan trọng là sự khác biệt giữa hai cách khai báo:

char *s = "Hello";   // string literal (chỉ đọc)
char arr[] = "Hello"; // mảng có thể sửa đổi

Trong trường hợp char *s, chuỗi được lưu trong vùng nhớ chỉ đọc (read-only). Nếu cố gắng sửa đổi:

s[0] = 'h'; // có thể gây lỗi runtime

chương trình có thể bị lỗi nghiêm trọng (segmentation fault).

Ngược lại, với char arr[], chuỗi được lưu trong stack và có thể thay đổi.

Một vấn đề nguy hiểm khi làm việc với con trỏ chuỗi là việc vượt quá giới hạn của chuỗi. Nếu con trỏ tiếp tục truy cập sau ký tự '\0', chương trình sẽ đọc dữ liệu không hợp lệ, dẫn đến lỗi hoặc lộ thông tin bộ nhớ (buffer over-read).

2.7 Tìm kiếm nhị phân (Binary Search)

2.7.1 Giới thiệu về tìm kiếm nhị phân

Trong lập trình, tìm kiếm (searching) là một thao tác rất phổ biến, nhằm xác định xem một phần tử có tồn tại trong một tập dữ liệu hay không. Một trong những thuật toán tìm kiếm hiệu quả nhất trên mảng là tìm kiếm nhị phân (binary search).

Tìm kiếm nhị phân là thuật toán tìm kiếm trên mảng đã được sắp xếp (sorted array). Ý tưởng chính của thuật toán là liên tục chia đôi không gian tìm kiếm để giảm số lượng phần tử cần kiểm tra.

Cụ thể, thay vì duyệt từng phần tử như tìm kiếm tuyến tính (linear search), thuật toán sẽ:

- So sánh giá trị cần tìm với phần tử ở giữa mảng
- Nếu bằng → tìm thấy

- Nếu nhỏ hơn → tiếp tục tìm ở nửa bên trái
- Nếu lớn hơn → tiếp tục tìm ở nửa bên phải

Quá trình này được lặp lại cho đến khi tìm thấy phần tử hoặc không còn phần tử nào để kiểm tra.

Xét ví dụ mảng đã sắp xếp:

[10, 22, 35, 47, 55, 68, 79]

Tìm giá trị 68:

- Bước 1: chọn phần tử giữa 47 → 68 > 47 → tìm bên phải
- Bước 2: xét đoạn [55, 68, 79], phần tử giữa là 68 → tìm thấy

Thuật toán tìm kiếm nhị phân có độ phức tạp là O(log n), nhanh hơn rất nhiều so với tìm kiếm tuyến tính O(n), đặc biệt với tập dữ liệu lớn.

Một cài đặt đơn giản bằng chỉ số:

```
int binary_search(int arr[], int n, int key) {
    int left = 0, right = n - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] > key)
            right = mid - 1;
        else
            left = mid + 1;
    }

    return -1;
}
```

2.7.2 Tìm kiếm nhị phân với con trỏ

Ngoài cách sử dụng chỉ số, tìm kiếm nhị phân còn có thể được cài đặt bằng con trỏ. Cách tiếp cận này cho phép thao tác trực tiếp trên địa chỉ bộ nhớ, và đặc biệt hữu ích khi làm việc với các cấu trúc dữ liệu phức tạp.

Ví dụ sử dụng con trỏ:

```
int* binary_search_ptr(int *low, int *high, int key) {
    if (low > high)
        return NULL;

    int *mid = low + (high - low) / 2;

    if (*mid == key)
        return mid;

    if (*mid > key)
        return binary_search_ptr(low, mid - 1, key);

    return binary_search_ptr(mid + 1, high, key);
}
```

Trong cách cài đặt này:

- low và high là hai con trỏ xác định phạm vi tìm kiếm
- (high - low) cho biết số phần tử giữa hai con trỏ
- mid được tính bằng phép toán trên con trỏ, không cần chỉ số

Một điểm quan trọng là việc truyền con trỏ giúp tránh sao chép mảng, do đó tiết kiệm bộ nhớ và tăng hiệu năng. Thực chất, thuật toán chỉ "dịch chuyển cửa sổ" trên cùng một vùng nhớ.

Tuy nhiên, cần đảm bảo rằng:

- Hai con trỏ luôn nằm trong cùng một mảng
- Không truy cập vượt quá giới hạn mảng (tránh undefined behavior)

2.8 Truyền tham số mảng

Trong C, khi truyền một mảng vào hàm, mảng sẽ tự động chuyển thành con trỏ (array decay). Điều này có nghĩa là hàm không nhận bản sao của toàn bộ mảng, mà chỉ nhận địa chỉ của phần tử đầu tiên.

Các cách khai báo tham số sau là tương đương:

```
void func(int *arr);
void func(int arr[]);
void func(int arr[10]);
```

Tất cả các khai báo trên đều được hiểu là con trỏ kiểu int*.

Xét ví dụ:

```
#include <stdio.h>

void modify(int arr[], int n) {
    arr[0] = 100;
}

int main() {
    int data[3] = {1, 2, 3};
    modify(data, 3);

    printf("%d\n", data[0]); // 100
    return 0;
}
```

Trong ví dụ này, việc thay đổi arr[0] bên trong hàm modify() đã ảnh hưởng trực tiếp đến mảng gốc. Điều này là do ta đang thao tác trên cùng một vùng nhớ.

Vấn đề với sizeof (sizeof trap)

Một lỗi phổ biến khi làm việc với mảng trong hàm là sử dụng toán tử sizeof để tính số phần tử.

```
void process(int arr[]) {
   int n = sizeof(arr) / sizeof(arr[0]); // sai
}
```

Trong hàm, arr thực chất là con trỏ, nên:

- sizeof(arr) trả về kích thước con trỏ (thường là 8 byte trên hệ 64-bit)
- Không phải kích thước của mảng

Do đó, kết quả sẽ sai.

Cách đúng là tính kích thước mảng trước khi truyền:

```
int arr[5] = {1, 2, 3, 4, 5};
int n = sizeof(arr) / sizeof(arr[0]);
process(arr, n);
```

Việc mảng được truyền dưới dạng con trỏ mang lại hai hệ quả quan trọng:

Thứ nhất, việc truyền mảng là hiệu quả về bộ nhớ, vì không cần sao chép toàn bộ dữ liệu.

Thứ hai, các thay đổi trong hàm sẽ ảnh hưởng trực tiếp đến dữ liệu gốc. Điều này rất hữu ích nhưng cũng tiềm ẩn nguy cơ nếu không kiểm soát tốt.

3. Cấp phát động (Dynamic Memory Allocation)

Trong các phần trước, chúng ta đã làm việc với biến cục bộ, mảng và con trỏ – những thành phần chủ yếu được cấp phát trên stack (ngăn xếp). Tuy nhiên, trong nhiều bài toán thực tế, kích thước dữ liệu không thể xác định trước tại thời điểm biên dịch hoặc cần tồn tại lâu hơn phạm vi của một hàm. Khi đó, việc sử dụng cấp phát bộ nhớ động (dynamic memory allocation) trên heap trở nên cần thiết.

Phần này sẽ trình bày các hạn chế của stack, giới thiệu các hàm cấp phát động trong C, cũng như các lỗi phổ biến liên quan đến quản lý bộ nhớ.

3.1 Giới hạn của stack và nhu cầu cấp phát động

Trong chương trình C, bộ nhớ được tổ chức thành nhiều vùng, trong đó hai vùng quan trọng là:

- Stack (ngăn xếp): dùng để lưu biến cục bộ và stack frame của hàm
- Heap (vùng nhớ động): dùng để cấp phát bộ nhớ trong thời gian chạy

Việc sử dụng stack có nhiều ưu điểm như nhanh và được quản lý tự động. Tuy nhiên, stack tồn tại một số hạn chế đáng kể.

Thứ nhất, kích thước của stack là giới hạn (thường khoảng vài MB). Nếu chương trình cấp phát quá nhiều dữ liệu trên stack, có thể xảy ra lỗi stack overflow.

```
int big[10000000]; // rất dễ gây stack overflow
```

Thứ hai, dữ liệu trên stack chỉ tồn tại trong phạm vi của hàm. Khi hàm kết thúc, toàn bộ biến cục bộ sẽ bị hủy.

```
int* createArray() {
   int arr[100];
   return arr; // lỗi: trả về địa chỉ vùng nhớ đã bị hủy
}
```

Trong ví dụ trên, con trỏ trả về trỏ đến vùng nhớ không còn hợp lệ (dangling pointer), gây lỗi nghiêm trọng khi sử dụng.

Thứ ba, kích thước dữ liệu trên stack thường phải được xác định trước khi chạy chương trình. Mặc dù C hỗ trợ mảng kích thước biến nhưng việc sử dụng không phải lúc nào cũng an toàn và không được khuyến khích trong lập trình hệ thống.

```
int n;
scanf("%d", &n);
int arr[n]; // phụ thuộc compiler, tiềm ẩn rủi ro
```

Những hạn chế trên dẫn đến nhu cầu sử dụng heap, nơi cho phép cấp phát bộ nhớ linh hoạt trong thời gian chạy và tồn tại cho đến khi được giải phóng.

3.2 Các hàm cấp phát động

Trong ngôn ngữ C, bộ nhớ trên heap không được cấp phát và giải phóng tự động như biến cục bộ trên stack. Thay vào đó, lập trình viên phải chủ động yêu cầu hệ điều hành cấp phát vùng nhớ khi cần và tự giải phóng vùng nhớ đó sau khi sử dụng xong. Để thực hiện việc này, C cung cấp một nhóm hàm trong thư viện <stdlib.h>, bao gồm malloc(), calloc(), realloc() và free().

Việc hiểu rõ từng hàm không chỉ dừng lại ở cú pháp, mà còn cần hiểu vùng nhớ được tạo ra như thế nào, dữ liệu ban đầu trong vùng nhớ đó ra sao, khi nào cần sử dụng từng hàm và những sai sót nào thường xuất hiện trong quá trình lập trình.

Hàm malloc()

Hàm malloc() là viết tắt của memory allocation, được dùng để cấp phát một vùng nhớ có kích thước xác định trên heap.

Cú pháp tổng quát của hàm là:

void *malloc(size_t size);

Trong đó, size là số byte cần cấp phát. Hàm trả về một con trỏ kiểu void *, tức là con trỏ chung có thể ép kiểu sang bất kỳ kiểu dữ liệu phù hợp nào.

Ví dụ, nếu muốn cấp phát vùng nhớ đủ để chứa 5 phần tử kiểu int, ta có thể viết:

int *arr = (int *) malloc(5 * sizeof(int));

Ở đây, sizeof(int) cho biết kích thước của một biến kiểu int trên hệ thống hiện tại. Việc dùng sizeof() là rất quan trọng, vì kích thước của kiểu dữ liệu có thể khác nhau giữa các hệ thống hoặc kiến trúc phần cứng. Nếu viết cứng giá trị 4 thay vì sizeof(int), chương trình sẽ kém an toàn và giảm tính khả chuyển.

Một điểm cần đặc biệt lưu ý là vùng nhớ do malloc() cấp phát không được khởi tạo. Điều này có nghĩa là nội dung trong vùng nhớ vừa cấp phát có thể là các giá trị rác còn sót lại từ những lần sử dụng trước của hệ thống. Vì vậy, sau khi cấp phát bằng malloc(), nếu muốn dùng dữ liệu một cách an toàn, ta cần tự gán giá trị cho từng phần tử.

Xét ví dụ sau:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *) malloc(n * sizeof(int));

    if (arr == NULL) {
        perror("malloc failed");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = i * 10;
    }
```

```
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
    arr = NULL;

    return 0;
}
```

Trong chương trình trên, malloc() cấp phát một vùng nhớ đủ chứa 5 số nguyên. Sau đó, ta kiểm tra xem con trỏ trả về có bằng NULL hay không. Điều này là cần thiết vì nếu hệ thống không đủ bộ nhớ, malloc() sẽ trả về NULL. Nếu không kiểm tra mà sử dụng ngay con trỏ này, chương trình có thể bị lỗi nghiêm trọng.

Tiếp theo, chương trình gán giá trị cho từng phần tử của mảng và in chúng ra màn hình. Cuối cùng, free(arr) được dùng để giải phóng vùng nhớ đã cấp phát, và arr = NULL giúp tránh việc vô tình sử dụng lại một con trỏ đã không còn hợp lệ.

Hàm calloc()

Hàm calloc() là viết tắt của contiguous allocation. Về chức năng, calloc() cũng dùng để cấp phát bộ nhớ trên heap, nhưng khác với malloc(), hàm này sẽ khởi tạo toàn bộ vùng nhớ về 0.

Cú pháp tổng quát:

void *calloc(size_t num, size_t size);

Trong đó:

- num là số phần tử cần cấp phát
- size là kích thước của mỗi phần tử

Ví dụ:

int *arr = (int *) calloc(5, sizeof(int));

Câu lệnh trên cấp phát vùng nhớ đủ chứa 5 số nguyên và đồng thời khởi tạo tất cả phần tử về giá trị 0.

Ta xét ví dụ đầy đủ:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *) calloc(n, sizeof(int));

    if (arr == NULL) {
        perror("calloc failed");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
    arr = NULL;

    return 0;
}
```

Trong ví dụ này, khi in ra các phần tử ngay sau khi cấp phát, kết quả sẽ là các số 0. Đây là ưu điểm lớn của calloc() trong những trường hợp ta muốn vùng nhớ ban đầu ở trạng thái xác định, thay vì chứa giá trị rác.

So với malloc(), calloc() thường được ưu tiên khi làm việc với mảng hoặc cấu trúc dữ liệu mà giá trị ban đầu nên là 0. Tuy nhiên, do phải thực hiện thêm bước khởi tạo, calloc() có thể tốn thêm một chút thời gian so với malloc().

Hàm realloc()

Trong nhiều trường hợp, sau khi đã cấp phát bộ nhớ, ta nhận ra kích thước ban đầu không còn phù hợp. Có thể dữ liệu cần lưu nhiều hơn dự kiến, hoặc ít hơn. Khi đó, ta cần thay đổi kích thước vùng nhớ đã cấp phát. Hàm realloc() được sử dụng cho mục đích này.

Cú pháp tổng quát:

void *realloc(void *ptr, size_t new_size);

Trong đó:

- ptr là con trỏ tới vùng nhớ đã được cấp phát trước đó bằng malloc(), calloc() hoặc realloc()
- new_size là kích thước mới mong muốn, tính theo byte

Ví dụ:

int *tmp = (int *) realloc(arr, 10 * sizeof(int));

Hàm realloc() có thể hoạt động theo hai cách. Nếu vùng nhớ hiện tại còn đủ không gian để mở rộng, hệ thống có thể giữ nguyên địa chỉ cũ và chỉ tăng kích thước vùng nhớ. Nếu không thể mở rộng tại chỗ, hệ thống sẽ cấp phát một vùng nhớ mới lớn hơn, sao chép dữ liệu cũ sang vùng mới, sau đó giải phóng vùng nhớ cũ và trả về địa chỉ mới.

Do đó, sau khi gọi realloc(), con trỏ trả về có thể giống hoặc khác con trỏ ban đầu.

Một điểm rất quan trọng là không nên viết trực tiếp:

arr = realloc(arr, 10 * sizeof(int));

Lý do là nếu realloc() thất bại, nó trả về NULL, trong khi con trỏ ban đầu arr vẫn còn trỏ tới vùng nhớ cũ. Nếu gán trực tiếp như trên, ta sẽ làm mất địa chỉ của vùng nhớ cũ, dẫn đến rò rỉ bộ nhớ. Vì vậy, cách an toàn là dùng biến trung gian:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *) malloc(n * sizeof(int));

    if (arr == NULL) {
```

```
    perror("malloc failed");
    return 1;
  }

  for (int i = 0; i < n; i++) {
    arr[i] = i * 10;
  }

  int *tmp = (int *) realloc(arr, 10 * sizeof(int));
  if (tmp == NULL) {
    free(arr);
    perror("realloc failed");
    return 1;
  }

  arr = tmp;

  for (int i = 5; i < 10; i++) {
    arr[i] = i * 10;
  }

  for (int i = 0; i < 10; i++) {
    printf("%d ", arr[i]);
  }
  printf("\n");

  free(arr);
  arr = NULL;

  return 0;
}
```

Trong ví dụ này, chương trình ban đầu cấp phát mảng 5 phần tử, sau đó dùng realloc() để mở rộng thành 10 phần tử. Dữ liệu cũ ở 5 phần tử đầu tiên vẫn được giữ nguyên. Tuy nhiên, các phần tử mới được thêm vào sau khi mở rộng không được khởi tạo, nên cần gán giá trị trước khi sử dụng.

Một trường hợp đặc biệt là khi ptr == NULL, lời gọi realloc(NULL, size) hoạt động tương tự như malloc(size). Ngoài ra, nếu new_size == 0, hành vi phụ thuộc hệ thống, nhưng về mặt thực hành, không nên lạm dụng trường hợp này để tránh code khó hiểu.

Hàm free()

Khi một vùng nhớ trên heap không còn cần thiết, ta phải giải phóng nó bằng hàm free() để trả bộ nhớ lại cho hệ thống.

Cú pháp:

void free(void *ptr);

Ví dụ:

```
free(arr);
arr = NULL;
```

Hàm free() không xóa biến con trỏ, mà chỉ giải phóng vùng nhớ mà con trỏ đó đang trỏ tới. Sau khi free(arr), biến arr vẫn tồn tại, nhưng địa chỉ mà nó đang giữ không còn hợp lệ nữa. Khi đó, arr trở thành một dangling pointer nếu vẫn còn giữ địa chỉ cũ.

Vì vậy, sau khi free(), ta thường gán con trỏ về NULL. Đây là một thói quen tốt trong lập trình C, vì:

- giúp tránh vô tình sử dụng lại vùng nhớ đã giải phóng
- giúp việc kiểm tra trạng thái con trỏ dễ dàng hơn

Ví dụ:

```
if (arr != NULL) {
   free(arr);
   arr = NULL;
}
```

Một nguyên tắc quan trọng cần ghi nhớ là: mỗi vùng nhớ được cấp phát thành công phải được giải phóng đúng một lần. Nếu giải phóng ít hơn, sẽ gây rò rỉ bộ nhớ. Nếu giải phóng nhiều hơn một lần, sẽ dẫn đến lỗi nghiêm trọng.

3.3 Các lỗi rò rỉ bộ nhớ

Khi làm việc với bộ nhớ động, việc cấp phát mới chỉ là một nửa của vấn đề. Nửa còn lại, và cũng là phần dễ sai nhất, là quản lý vòng đời của vùng nhớ đã cấp phát. Nếu không

giải phóng đúng cách, chương trình sẽ gặp lỗi rò rỉ bộ nhớ (memory leak) hoặc nhiều lỗi nghiêm trọng khác liên quan đến con trỏ.

Rò rỉ bộ nhớ (memory leak) xảy ra khi chương trình đã cấp phát một vùng nhớ trên heap nhưng sau đó làm mất khả năng truy cập tới vùng nhớ đó mà không giải phóng nó. Khi điều này xảy ra, vùng nhớ vẫn chiếm tài nguyên hệ thống nhưng chương trình không còn cách nào hợp lệ để sử dụng hay thu hồi nó nữa.

Xét ví dụ đơn giản sau:

```
#include <stdlib.h>

void bad() {
    int *p = (int *) malloc(10 * sizeof(int));
    if (p == NULL) {
        return;
    }

    p[0] = 100;
    /* quên free(p); */
}
```

Trong hàm bad(), vùng nhớ đã được cấp phát thành công cho p. Tuy nhiên, trước khi hàm kết thúc, chương trình không gọi free(p). Khi đó, biến p bị hủy do ra khỏi phạm vi hàm, nhưng vùng nhớ trên heap vẫn còn tồn tại. Vì không còn con trỏ nào trỏ đến vùng nhớ đó nữa, nó trở thành vùng nhớ bị mất.

Trong các chương trình ngắn, lỗi này có thể không gây hậu quả dễ thấy ngay lập tức vì chương trình kết thúc sớm và hệ điều hành sẽ thu hồi bộ nhớ. Tuy nhiên, trong các chương trình chạy lâu như server, ứng dụng nhúng hoặc chương trình xử lý dữ liệu lớn, rò rỉ bộ nhớ sẽ khiến lượng RAM bị chiếm dụng tăng dần theo thời gian, cuối cùng làm chậm hệ thống hoặc khiến chương trình không thể cấp phát thêm bộ nhớ mới.

Nguyên nhân phổ biến gây memory leak

Một trong những nguyên nhân đơn giản nhất là quên gọi free() sau khi sử dụng xong bộ nhớ.

Ví dụ:

```
int *p = (int *) malloc(100 * sizeof(int));
/* dùng p */
/* quên free(p) */
```

Nguyên nhân thứ hai là ghi đè con trỏ trước khi giải phóng vùng nhớ cũ. Đây là lỗi rất hay gặp ở người mới học.

Ví dụ:

```
int *p = (int *) malloc(40);
p = (int *) malloc(80);
```

Ở đây, sau dòng lệnh thứ hai, địa chỉ vùng nhớ 40 byte ban đầu đã bị mất hoàn toàn. Con trỏ p bây giờ trỏ đến vùng nhớ mới 80 byte, còn vùng nhớ cũ không còn cách nào truy cập để giải phóng. Kết quả là vùng nhớ cũ bị rò rỉ.

Nguyên nhân thứ ba là thoát khỏi hàm quá sớm bằng return, break hoặc các nhánh lỗi mà quên giải phóng tài nguyên.

Ví dụ:

```
int *p = (int *) malloc(100 * sizeof(int));
if (p == NULL) {
   return;
}

if (some_error) {
   return;   /* quên free(p) */
}
```

Trong trường hợp này, nếu some_error xảy ra, hàm kết thúc ngay nhưng vùng nhớ của p chưa được giải phóng.

Nguyên nhân thứ tư là cấp phát trong vòng lặp nhưng không giải phóng trong mỗi lần lặp.

Ví dụ:

```
for (int i = 0; i < 1000; i++) {
   int *p = (int *) malloc(100 * sizeof(int));
   /* xử lý dữ liệu */
}
```

Nếu trong vòng lặp trên không có free(p), mỗi lần lặp chương trình lại làm mất thêm một vùng nhớ. Đây là kiểu lỗi rất nguy hiểm vì lượng bộ nhớ bị mất tăng nhanh theo số vòng lặp.

Ví dụ về cách viết đúng

Để tránh rò rỉ bộ nhớ, ta cần đảm bảo rằng mọi đường đi hợp lệ trong chương trình đều dẫn đến việc giải phóng vùng nhớ đã cấp phát.

Ví dụ:

```
#include <stdio.h>
#include <stdlib.h>

void good() {
    int *p = (int *) malloc(10 * sizeof(int));
    if (p == NULL) {
        return;
    }

    for (int i = 0; i < 10; i++) {
        p[i] = i;
    }

    free(p);
    p = NULL;
}
```

Trong ví dụ này, vùng nhớ được cấp phát, sử dụng, sau đó giải phóng đúng cách. Việc gán p = NULL sau free() giúp hạn chế nguy cơ sử dụng lại con trỏ cũ.

Các lỗi nghiêm trọng liên quan đến bộ nhớ

Ngoài memory leak, còn có một số lỗi khác cũng rất thường gặp khi làm việc với heap. Mặc dù chúng không hoàn toàn giống rò rỉ bộ nhớ, nhưng đều là hậu quả của việc quản lý bộ nhớ không đúng cách.

Dangling pointer

Dangling pointer là con trỏ vẫn giữ địa chỉ của một vùng nhớ đã bị giải phóng hoặc không còn hợp lệ.

Ví dụ:

```
int *p = (int *) malloc(sizeof(int));
free(p);
/* p vẫn đang chứa địa chỉ cũ */
```

Sau khi free(p), biến p vẫn còn tồn tại nhưng địa chỉ mà nó chứa không còn hợp lệ. Nếu tiếp tục dùng p, chương trình có thể truy cập vào vùng nhớ đã được hệ thống cấp cho mục đích khác, gây ra lỗi khó lường.

Use-after-free

Use-after-free là lỗi xảy ra khi chương trình truy cập vùng nhớ sau khi đã giải phóng nó.

Ví dụ:

```
int *p = (int *) malloc(sizeof(int));
free(p);
*p = 10;
```

Đây là một lỗi rất nguy hiểm. Chương trình có thể bị crash ngay, nhưng cũng có thể vẫn chạy và cho ra kết quả sai, khiến lỗi trở nên rất khó phát hiện.

Double free

Double free là lỗi giải phóng cùng một vùng nhớ nhiều hơn một lần.

Ví dụ:

```
int *p = (int *) malloc(sizeof(int));
free(p);
free(p);
```

Hành vi này là undefined behavior. Trên nhiều hệ thống, nó có thể làm hỏng bộ quản lý heap của chương trình và gây crash ngay lập tức.

Dereference NULL pointer

Một lỗi khác là truy cập vào con trỏ NULL.

Ví dụ:

```
int *p = NULL;
*p = 5;
```

Vì NULL không trỏ đến vùng nhớ hợp lệ nào, thao tác này sẽ làm chương trình bị lỗi nghiêm trọng.

4.2 Giới thiệu về danh sách liên kết

Để xây dựng được danh sách liên kết trong ngôn ngữ C, trước hết cần hiểu rõ vai trò của cấu trúc (struct). Trong C, struct là một kiểu dữ liệu do người lập trình tự định nghĩa, cho phép gom nhiều thành phần dữ liệu liên quan lại với nhau trong cùng một đơn vị. Mỗi thành phần trong struct được gọi là một trường (field).

Ví dụ, nếu muốn biểu diễn thông tin của một điểm trong mặt phẳng, ta có thể định nghĩa:

```
struct Point {
   int x;
   int y;
};
```

Trong cấu trúc trên, x và y là hai trường dữ liệu, cùng thuộc về một đối tượng kiểu struct Point. Cách tổ chức này giúp dữ liệu rõ ràng hơn, dễ quản lý hơn và thuận tiện khi xây dựng các cấu trúc dữ liệu phức tạp.

Khi làm việc với danh sách liên kết, ta cần biểu diễn mỗi phần tử của danh sách như một nút dữ liệu. Mỗi nút không chỉ cần lưu giá trị của chính nó mà còn phải biết nút tiếp theo nằm ở đâu trong bộ nhớ. Vì vậy, ta cần một cấu trúc gồm hai thành phần: một trường để lưu dữ liệu và một trường con trỏ để liên kết với nút kế tiếp.

Cấu trúc cơ bản của một nút trong danh sách liên kết thường được định nghĩa như sau:

```
struct Node {
   int data;
   struct Node *next;
};
```

Cấu trúc này rất quan trọng vì nó thể hiện đúng bản chất của danh sách liên kết. Trường data dùng để lưu giá trị của nút, còn trường next là một con trỏ kiểu struct Node *, dùng để lưu địa chỉ của nút tiếp theo trong danh sách. Nói cách khác, mỗi nút vừa chứa dữ liệu, vừa chứa “liên kết” đến nút kế tiếp.

Một điểm đáng chú ý ở đây là struct Node chứa một con trỏ trỏ tới chính kiểu struct Node. Đây được gọi là cấu trúc tự tham chiếu (self-referential structure). Kiểu cấu trúc này là nền tảng để xây dựng các cấu trúc dữ liệu liên kết như linked list, tree hay graph.

Để dễ hình dung, ta xét một danh sách liên kết gồm ba nút:

10 -> 20 -> 30 -> NULL

Danh sách trên có thể được hiểu như sau. Nút đầu tiên chứa giá trị 10, và trường next của nó trỏ đến nút chứa giá trị 20. Nút chứa 20 lại trỏ đến nút chứa 30. Nút cuối cùng có next = NULL, nghĩa là không còn nút nào phía sau nữa. Giá trị NULL trong C biểu diễn con trỏ rỗng, tức là không trỏ đến vùng nhớ hợp lệ nào.

Để truy cập vào toàn bộ danh sách, ta chỉ cần giữ con trỏ tới nút đầu tiên. Con trỏ này thường được gọi là head. Từ head, ta có thể lần lượt đi theo các liên kết next để duyệt toàn bộ danh sách.

Ví dụ, nếu head trỏ đến nút đầu tiên, ta có thể hình dung như sau:

```
head
 |
 v
10 -> 20 -> 30 -> NULL
```

Trong danh sách liên kết đơn (singly linked list), mỗi nút chỉ biết nút tiếp theo, chứ không biết nút đứng trước nó. Vì vậy, việc duyệt danh sách luôn bắt đầu từ head và đi theo một chiều từ trái sang phải.

So với mảng, danh sách liên kết có cách tổ chức bộ nhớ hoàn toàn khác. Trong mảng, các phần tử được lưu trong các ô nhớ liên tiếp nhau. Điều này cho phép truy cập nhanh bằng chỉ số, nhưng lại gây khó khăn khi cần chèn hoặc xóa phần tử ở giữa. Ngược lại, trong danh sách liên kết, các nút có thể nằm ở những vị trí hoàn toàn khác nhau trong bộ nhớ heap, và được nối với nhau bằng các con trỏ. Nhờ đó, ta không cần một vùng nhớ liên tiếp lớn, và việc mở rộng danh sách trở nên linh hoạt hơn.

Tuy nhiên, chính vì các phần tử không nằm liên tiếp nên danh sách liên kết không hỗ trợ truy cập ngẫu nhiên (random access) như mảng. Nếu muốn đến phần tử thứ ba hay thứ tư, ta không thể viết như arr[3], mà phải bắt đầu từ head, sau đó lần lượt đi qua từng nút trung gian.

Trong thực hành, một thao tác rất phổ biến là tạo một nút mới bằng cấp phát động. Khi đó, ta thường viết một hàm hỗ trợ như sau:

```
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    if (n == NULL) {
        return NULL;
    }

    n->data = val;
    n->next = NULL;
    return n;
}
```

Hàm newNode() trên thực hiện ba công việc. Trước hết, hàm dùng malloc() để cấp phát bộ nhớ trên heap cho một nút mới. Sau đó, trường data của nút được gán bằng giá trị val. Cuối cùng, trường next được khởi tạo bằng NULL, vì tại thời điểm mới tạo, nút này chưa liên kết với nút nào khác.

Việc tách thao tác tạo nút thành một hàm riêng có ý nghĩa rất lớn trong thực hành. Nó giúp mã nguồn rõ ràng hơn, tránh lặp lại code nhiều lần và giúp việc kiểm tra lỗi cấp phát bộ nhớ trở nên tập trung hơn. Trong các chương trình lớn, đây là một thói quen tốt nên duy trì.

Như vậy, có thể hiểu danh sách liên kết là một tập hợp các nút được cấp phát động trên heap, trong đó mỗi nút chứa dữ liệu và địa chỉ của nút kế tiếp. Toàn bộ danh sách được quản lý thông qua con trỏ head. Đây là cấu trúc dữ liệu rất quan trọng trong lập trình C, vì nó thể hiện sự kết hợp giữa struct, con trỏ và cấp phát động – ba nội dung cốt lõi của lập trình hệ thống.

4.3 Các thao tác trên danh sách liên kết

Sau khi đã hiểu cách tổ chức của một danh sách liên kết, bước tiếp theo là thực hiện các thao tác cơ bản trên cấu trúc dữ liệu này. Trong phần này, ta sẽ lần lượt xét các thao tác

thường gặp như thêm phần tử mới, kiểm tra phần tử đầu danh sách, in danh sách, tìm kiếm và giải phóng bộ nhớ. Với mỗi thao tác, cần hiểu không chỉ cú pháp code mà còn phải hiểu rõ cơ chế hoạt động của con trỏ trong từng dòng lệnh.

Thêm phần tử mới

Một trong những thao tác đơn giản và quan trọng nhất là thêm phần tử mới vào đầu danh sách. Đây là cách chèn phổ biến vì thao tác này không cần duyệt qua các nút hiện có.

Ví dụ:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    if (n == NULL) {
        return NULL;
    }
    n->data = val;
    n->next = NULL;
    return n;
}

void insertHead(struct Node **head, int val) {
    struct Node *n = newNode(val);
    if (n == NULL) {
        return;
    }
    n->next = *head;
    *head = n;
}
```

Để hiểu hàm insertHead(), trước hết cần chú ý rằng tham số của hàm là struct Node **head, tức là con trỏ tới con trỏ đầu danh sách. Lý do là vì thao tác chèn vào đầu danh sách có thể làm thay đổi chính biến head ở bên ngoài hàm. Nếu chỉ truyền struct Node *head, ta chỉ thay đổi được bản sao cục bộ của head, chứ không thể cập nhật danh sách thật trong hàm gọi.

Bên trong hàm, câu lệnh:

struct Node *n = newNode(val);

tạo ra một nút mới có giá trị val.

Tiếp theo, câu lệnh:

n->next = *head;

gắn trường next của nút mới vào nút đầu hiện tại của danh sách. Nói cách khác, nút mới sẽ đứng trước toàn bộ danh sách cũ.

Cuối cùng, câu lệnh:

*head = n;

cập nhật head để nó trỏ đến nút mới. Sau bước này, nút mới trở thành phần tử đầu tiên của danh sách.

Ví dụ, nếu trước đó danh sách là:

10 -> 20 -> 30 -> NULL

và ta chèn thêm giá trị 5 vào đầu, quá trình sẽ diễn ra như sau:

- tạo nút mới chứa 5
- cho next của nút mới trỏ vào nút chứa 10
- cập nhật head để trỏ vào nút 5

Kết quả là:

5 -> 10 -> 20 -> 30 -> NULL

Thao tác này có độ phức tạp thời gian là O(1), vì số lượng bước thực hiện là cố định, không phụ thuộc vào số phần tử của danh sách. Dù danh sách có 3 phần tử hay 3000 phần tử, ta vẫn chỉ cần tạo một nút mới và thay đổi một vài con trỏ.

Kiểm tra một phần tử có phải đầu danh sách hay không

Trong danh sách liên kết, phần tử đầu tiên luôn là nút mà head đang trỏ tới. Do đó, nếu ta đã có một con trỏ p trỏ đến một nút nào đó, việc kiểm tra nút đó có phải là đầu danh sách hay không chỉ cần so sánh địa chỉ:

```
if (p == head) {
   printf("p la nut dau danh sach\n");
}
```

Phép so sánh này không so sánh dữ liệu bên trong nút mà so sánh địa chỉ bộ nhớ. Nếu p và head cùng trỏ tới một vùng nhớ, nghĩa là chúng cùng chỉ vào một nút, khi đó p chính là nút đầu danh sách.

Cần phân biệt rõ điều này với việc so sánh dữ liệu. Ví dụ, nếu có hai nút khác nhau nhưng cùng chứa giá trị 10, thì p->data == head->data chưa đủ để kết luận p là đầu danh sách. Điều quyết định ở đây là con trỏ, không phải giá trị dữ liệu.

Thao tác kiểm tra này có độ phức tạp O(1), vì chỉ cần thực hiện một phép so sánh duy nhất giữa hai địa chỉ.

In danh sách liên kết

Để in toàn bộ danh sách liên kết, ta cần duyệt từng nút bắt đầu từ head, sau đó đi theo con trỏ next cho đến khi gặp NULL.

Ví dụ:

```
void printList(struct Node *head) {
   while (head != NULL) {
      printf("%d -> ", head->data);
      head = head->next;
   }
   printf("NULL\n");
}
```

Trong hàm này, biến head đóng vai trò như một con trỏ duyệt tạm thời. Ở mỗi vòng lặp, chương trình thực hiện hai việc:

- in giá trị của nút hiện tại bằng head->data
- dịch sang nút kế tiếp bằng head = head->next

Vòng lặp dừng lại khi head == NULL, tức là đã đi đến cuối danh sách.

Ví dụ, với danh sách:

5 -> 10 -> 20 -> NULL

quá trình duyệt sẽ như sau:

- ban đầu head trỏ tới nút 5, in 5
- sau đó head chuyển sang nút 10, in 10
- tiếp tục chuyển sang nút 20, in 20
- sau đó head = NULL, kết thúc

Kết quả in ra:

5 -> 10 -> 20 -> NULL

Độ phức tạp của thao tác in danh sách là O(n), trong đó n là số phần tử của danh sách. Lý do là để in được toàn bộ danh sách, ta phải đi qua từng nút đúng một lần. Số lần lặp tăng tuyến tính theo số phần tử.

Tìm kiếm trên danh sách liên kết

Do danh sách liên kết không hỗ trợ truy cập ngẫu nhiên, thao tác tìm kiếm phải được thực hiện bằng cách duyệt tuần tự từ đầu đến cuối.

Ví dụ:

```
struct Node *search(struct Node *head, int key) {
    while (head != NULL) {
        if (head->data == key) {
            return head;
        }
        head = head->next;
    }
    return NULL;
}
```

Hàm search() bắt đầu từ nút đầu tiên. Ở mỗi bước, chương trình so sánh giá trị head->data với khóa cần tìm key. Nếu trùng, hàm trả về địa chỉ của nút hiện tại. Nếu không

trùng, hàm dịch sang nút kế tiếp. Nếu duyệt hết danh sách mà không tìm thấy, hàm trả về NULL.

Ví dụ, với danh sách:

5 -> 10 -> 20 -> 30 -> NULL

khi tìm 20, chương trình sẽ:

- so sánh với 5, không khớp
- so sánh với 10, không khớp
- so sánh với 20, khớp, trả về nút chứa 20

Nếu tìm 99, chương trình phải duyệt hết toàn bộ danh sách rồi mới kết luận là không tồn tại.

Độ phức tạp của thao tác tìm kiếm là O(n) trong trường hợp xấu nhất. Điều này là do phần tử cần tìm có thể nằm ở cuối danh sách, hoặc không tồn tại, khi đó phải duyệt qua tất cả các nút. Trong trường hợp tốt nhất, nếu phần tử nằm ngay ở đầu danh sách, thời gian có thể là O(1). Tuy nhiên, khi phân tích độ phức tạp, ta thường quan tâm đến trường hợp xấu nhất nên kết luận là O(n).

Giải phóng danh sách liên kết

Do các nút trong danh sách được cấp phát động bằng malloc(), sau khi sử dụng xong, ta phải giải phóng từng nút để tránh rò rỉ bộ nhớ (memory leak).

Ví dụ:

```
void freeList(struct Node *head) {
   while (head != NULL) {
      struct Node *tmp = head;
      head = head->next;
      free(tmp);
   }
}
```

Đây là một thao tác rất quan trọng trong thực hành. Nếu không giải phóng danh sách, chương trình sẽ để lại các vùng nhớ không còn sử dụng nhưng vẫn chiếm chỗ trên heap.

Cách hoạt động của hàm freeList() cần được hiểu thật cẩn thận. Ở mỗi vòng lặp:

- tmp = head lưu lại địa chỉ của nút hiện tại
- head = head->next chuyển sang nút kế tiếp
- free(tmp) giải phóng nút cũ

Thứ tự này là bắt buộc. Nếu gọi free(head) trước rồi mới truy cập head->next, chương trình sẽ truy cập vào vùng nhớ đã bị giải phóng, dẫn đến lỗi use-after-free.

Ví dụ, với danh sách:

5 -> 10 -> 20 -> NULL

quá trình giải phóng diễn ra như sau:

- lưu nút 5 vào tmp, chuyển head sang 10, giải phóng 5
- lưu nút 10 vào tmp, chuyển head sang 20, giải phóng 10
- lưu nút 20 vào tmp, chuyển head sang NULL, giải phóng 20

Khi vòng lặp kết thúc, toàn bộ danh sách đã được thu hồi.

Độ phức tạp của thao tác giải phóng danh sách là O(n), vì phải đi qua và giải phóng từng nút một. Không có cách nào giải phóng toàn bộ danh sách trong O(1), do mỗi nút là một vùng nhớ cấp phát riêng biệt trên heap.